

# Programación Dinámica – Parte II

TTPS: Técnicas y Estrategias para la Resolución de Problemas

JTP - Matías Fluxa

Universidad Nacional de La Plata

14 de septiembre de 2026

## ① Knapsack Problem

Primer enfoque: Greedy

Segundo enfoque: Programación Dinámica

Conclusiones

Resolución con DP paso a paso

## ② ¿Dónde practicar más problemas de DP?

# Knapsack Problem: Enunciado

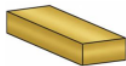
Dado un conjunto de  $n$  artículos, donde cada artículo  $i$  tiene un peso  $P_i$  y un valor  $V_i$ , y dada una capacidad máxima  $W$  para una mochila.

Determinar qué subconjunto de artículos se debe incluir para que:

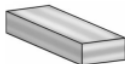
- La suma total de los pesos no exceda la capacidad máxima  $W$ .
- La suma total de los valores sea la máxima posible.

# Knapsack Problem

$$P_1 = 2 \text{ kg} - V_1 = 600$$



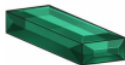
$$P_2 = 4 \text{ kg} - V_2 = 1000$$



$$P_3 = 5 \text{ kg} - V_3 = 1300$$



$$P_4 = 3 \text{ kg} - V_4 = 500$$



$$P_5 = 1 \text{ kg} - V_5 = 250$$



Capacidad  $W = 10 \text{ kg}$



# Knapsack Problem - Primer enfoque

Ordenamos los ítems por “Densidad”

| Ítem         | Peso [kg] | Valor [\$] | Densidad [\$ /kg] |
|--------------|-----------|------------|-------------------|
| 1. Oro       | 2         | 600        | 300               |
| 3. Bronce    | 5         | 1300       | 260               |
| 2. Plata     | 4         | 1000       | 250               |
| 5. Vidrio    | 1         | 250        | 250               |
| 4. Esmeralda | 3         | 500        | 167               |

# Knapsack Problem - Primer enfoque

Si elegimos los objetos del más al menos denso, obtenemos:

Oro + Bronce + Vidrio

Peso total: 8 *kg*      Valor: 2,150

# Knapsack Problem - Primer enfoque

Si elegimos los objetos del más al menos denso, obtenemos:

Oro + Bronce + Vidrio

Peso total: 8 kg      Valor: 2,150

Pero esta configuración no es la más óptima.

Necesitamos explorar las combinaciones posibles de manera sistemática.

## Estado

**$dp[i][x]$**  : máximo valor posible utilizando los primeros  $i$  ítems  
y exactamente  $x$  kilos.

## Estado

**$dp[i][x]$**  : máximo valor posible utilizando los primeros  $i$  ítems  
y exactamente  $x$  kilos.

Supongamos que ya conocemos las soluciones óptimas para los primeros  $i - 1$  ítems. Si procesamos el  $i$ -ésimo ítem, obtenemos:

$$\mathbf{dp[i][x] = \max(dp[i - 1][x], dp[i - 1][x - P_i] + V_i) .}$$

# Knapsack Problem - Programación Dinámica

Tenemos dos posibilidades para el objeto  $i$ : usarlo o no usarlo.

Transición 1: no usamos el objeto  $i$

$$\mathbf{dp}[i][x] = \mathbf{dp}[i - 1][x]$$

Nos quedamos con la mejor configuración para  $i - 1$  objetos.

Transición 2: usamos el objeto  $i$

$$\mathbf{dp}[i][x] = \mathbf{dp}[i - 1][x - P_i] + V_i$$

Antes debíamos tener una configuración de  $i - 1$  objetos que pesara  $x - P_i$ .

- No importa el orden en que se procesan los objetos: cualquier permutación permite encontrar la solución correcta.
- La complejidad es  $O(N \cdot W)$ , donde  $N$  es la cantidad de ítems y  $W$  la capacidad máxima.
- También existen soluciones con complejidad  $O\left(N \cdot \sum_{i=1}^N V_i\right)$ .
- Puede implementarse utilizando solo  $O(W)$  de memoria.

# Resolución con DP

Paso a paso la resolución de los estados

Representamos con  $-1$  los estados no posibles.

| $i \backslash x$ | 0 | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |
|------------------|---|----|----|----|----|----|----|----|----|----|----|
| 0                | 0 | -1 | -1 | -1 | -1 | -1 | -1 | -1 | -1 | -1 | -1 |

# Resolución con DP

Paso a paso la resolución de los estados

Representamos con  $-1$  los estados no posibles.

| $i \backslash x$ | 0 | 1  | 2   | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |
|------------------|---|----|-----|----|----|----|----|----|----|----|----|
| 0                | 0 | -1 | -1  | -1 | -1 | -1 | -1 | -1 | -1 | -1 | -1 |
| 1                | 0 | -1 | 600 | -1 | -1 | -1 | -1 | -1 | -1 | -1 | -1 |

# Resolución con DP

Paso a paso la resolución de los estados

Representamos con  $-1$  los estados no posibles.

| $i \backslash x$ | 0 | 1  | 2   | 3  | 4    | 5  | 6    | 7  | 8  | 9  | 10 |
|------------------|---|----|-----|----|------|----|------|----|----|----|----|
| 0                | 0 | -1 | -1  | -1 | -1   | -1 | -1   | -1 | -1 | -1 | -1 |
| 1                | 0 | -1 | 600 | -1 | -1   | -1 | -1   | -1 | -1 | -1 | -1 |
| 2                | 0 | -1 | 600 | -1 | 1000 | -1 | 1600 | -1 | -1 | -1 | -1 |

# Resolución con DP

Paso a paso la resolución de los estados

Representamos con  $-1$  los estados no posibles.

| $i \backslash x$ | 0 | 1  | 2   | 3  | 4    | 5    | 6    | 7    | 8  | 9    | 10 |
|------------------|---|----|-----|----|------|------|------|------|----|------|----|
| 0                | 0 | -1 | -1  | -1 | -1   | -1   | -1   | -1   | -1 | -1   | -1 |
| 1                | 0 | -1 | 600 | -1 | -1   | -1   | -1   | -1   | -1 | -1   | -1 |
| 2                | 0 | -1 | 600 | -1 | 1000 | -1   | 1600 | -1   | -1 | -1   | -1 |
| 3                | 0 | -1 | 600 | -1 | 1000 | 1300 | 1600 | 1900 | -1 | 2300 | -1 |

# Resolución con DP

Paso a paso la resolución de los estados

Representamos con  $-1$  los estados no posibles.

| $i \backslash x$ | 0 | 1  | 2   | 3   | 4    | 5    | 6    | 7    | 8    | 9    | 10   |
|------------------|---|----|-----|-----|------|------|------|------|------|------|------|
| 0                | 0 | -1 | -1  | -1  | -1   | -1   | -1   | -1   | -1   | -1   | -1   |
| 1                | 0 | -1 | 600 | -1  | -1   | -1   | -1   | -1   | -1   | -1   | -1   |
| 2                | 0 | -1 | 600 | -1  | 1000 | -1   | 1600 | -1   | -1   | -1   | -1   |
| 3                | 0 | -1 | 600 | -1  | 1000 | 1300 | 1600 | 1900 | -1   | 2300 | -1   |
| 4                | 0 | -1 | 600 | 500 | 1000 | 1300 | 1600 | 1900 | 1800 | 2300 | 2400 |

# Resolución con DP

Paso a paso la resolución de los estados

Representamos con  $-1$  los estados no posibles.

| $i \backslash x$ | 0 | 1   | 2   | 3   | 4    | 5    | 6    | 7    | 8    | 9    | 10   |
|------------------|---|-----|-----|-----|------|------|------|------|------|------|------|
| 0                | 0 | -1  | -1  | -1  | -1   | -1   | -1   | -1   | -1   | -1   | -1   |
| 1                | 0 | -1  | 600 | -1  | -1   | -1   | -1   | -1   | -1   | -1   | -1   |
| 2                | 0 | -1  | 600 | -1  | 1000 | -1   | 1600 | -1   | -1   | -1   | -1   |
| 3                | 0 | -1  | 600 | -1  | 1000 | 1300 | 1600 | 1900 | -1   | 2300 | -1   |
| 4                | 0 | -1  | 600 | 500 | 1000 | 1300 | 1600 | 1900 | 1800 | 2300 | 2400 |
| 5                | 0 | 250 | 600 | 850 | 1000 | 1300 | 1600 | 1900 | 2150 | 2300 | 2550 |

La mejor solución para una capacidad de 10 *kg* es  $\mathbf{dp}[5][10] = 2550$ .

# ¿Dónde practicar más problemas de DP?

La programación dinámica es una técnica muy evaluada en muchas entrevistas laborales. Dominarla puede ser determinante para obtener un puesto laboral.

La variedad de problemas es infinita, hay que comprender bien la técnica y no memorizar cada ejemplo.

La mejor forma de practicar es hacer muchos ejercicios, comprender su lógica y retenerla para futuros problemas.

# ¿Dónde practicar más problemas de DP?

- **LeetCode:**

<https://leetcode.com/problem-list/dynamic-programming/>  
Gran página para practicar problemas de entrevistas y tutoriales de varias técnicas.

- **Guía de DP de AtCoder:**

<https://atcoder.jp/contests/dp/tasks>  
Problemas desde la A hasta la Z, con un gran rango de problemas y ramas de aplicación.

- **Guía de DP de CSES:** <https://cses.fi/problemset/>  
Problemas típicos de varias ramas, incluidos ejercicios clásicos de programación dinámica.

En la final mundial de programación ICPC tomaron un problema que se resuelve con programación dinámica. Era uno de los problemas más sencillos de la prueba, pero aun así fue lindo y desafiante. Solamente un equipo de 139 no pudo resolverlo.

**¡El primer equipo en resolverlo tardó 14 minutos!**

<https://worldfinals.icpc.global/problems/2025/finals/problems/D-buggyrover.pdf>

No era el único problema de la prueba que utilizaba DP.